The browser session

A `Session` is one open document in one browser tab: the copy people edit, the queue of changes on their way to the server, the stream of changes coming back, the undo stack and the evaluated values. This page describes how an edit is applied at once and confirmed later, how the pending queue is rebased on what the server confirms, how undo works, and how the browser runs handlers by itself.

It describes session.ts, which builds on ops.ts, engine.ts and events.ts, and sends through transport.ts.

Two copies of the document

A session holds the document twice.

```TypeScript
private confirmed: Doc | null = null;   // what the server has told it
private pending: Pending[] = [];         // changes made here, not yet confirmed
// state.doc                            // what the person sees: confirmed + pending
```

`confirmed` is the last version the server has said is real. `state.doc` is `confirmed` with every pending change applied on top. Components read `state.doc`, so a change shows the moment it is made, with no wait for the network. The pending list is what makes that safe to do: the session always knows which changes the server has not yet agreed to.

```TypeScript
interface Pending {
  batch: string;
  ops: Op[];
  state: 'queued' | 'sent' | 'done';
  v?: number;
}
```

A `Pending` is one *batch*, a list of ops sent in one request. `queued` means not sent yet, `sent` that the request is out, and `done` that the server answered and the batch waits for the stream to say the same.

The session also keeps the evaluated document (`computed`, from The engine), the selection, the mode (**Edit**, **Live** or **Page**), the activity log, the people present, the records of the collections the document reads, the fetch states, and the trace of handlers that ran. It is a plain store (`createStore`) that components subscribe to with `useS`.

Opening

`open()` asks, in this order:

1. GET /api/library/:id whether the document exists at all, so a deleted or unreachable one asks for nothing else.
2. In parallel: the document (GET /api/docs/:id), its recent log, its messages, its fetch states and, when signed in, who may open it and this person's role.
3. It stores the document as `confirmed`, sets the role (someone who may only read starts in Live), and opens the event stream.
4. It loads the collections the document's handlers use, so the document's `open` handler sees the same records the server would. Then `opening()` fires `open`, once, the first time the session is in Live.

A guest who came in with a share link skips the library check, the log and the messages.

dispatch: making a change

Every change a person makes, whether typing in a cell, choosing a kind or dragging a divider, goes through `session.dispatch(op | ops, options)`. It does six things:

1. **React.** In Live, unless `options.react` is false, `react(doc, world, { ops })` from `events.ts` runs the handlers the change sets off. Their ops are appended to the same list, so the gesture and what it triggers are one step.
2. **Apply.** `applyOps(doc, ops)` runs here, in the browser. If it throws, a toast shows the message and nothing else happens.
3. **Normalize for sending.** The ops that go to the server are the *normalized* ones `applyOp` returned, with every id filled in (see Applying ops); `do` is unwrapped.
4. **Push undo.** Unless `undo` is false, the inverse op goes on the undo stack.
5. **Queue.** The ops join the last pending batch if it is still `queued`, merged by `coalesce()`; otherwise they start a new batch.
6. **Publish and flush.** The new document is published to the store, which evaluates it and re-renders, and `flush()` starts sending.

`coalesce` is how typing stays cheap. It replaces a `set` of the same property of the same cell by the later one, but only while the batch hasn't been sent. Ten keystrokes in an input make one `set`, not ten.

The queue and flush

`flush()` sends one batch at a time. It takes the first `queued` batch, marks it `sent` and posts it:

```TypeScript
api('POST', `/api/docs/${this.id}/ops`, { ops: next.ops, actor: this.actor, client: this.client, batch: next.batch })
```

The route is POST /api/docs/:id/ops. `client` identifies this tab and `batch` this batch. Both come back on the `ops` event, which is how the sender recognises its own change in the stream.

When the answer arrives the batch becomes `done` and records the version `v` the server gave it. The server's `ops` event normally follows within milliseconds. If the session still hasn't seen version `v` after a second, it assumes the stream missed something, reconnects and resyncs.

What happens on a failure depends on its kind:

The request	What the session does
Fails with a status below 500 (the server refused: bad op, no permission)	Drops the batch, shows `That change was not saved.` with the message, and rebases without it.
Fails with a 5xx, or the network is down	Sets `online` to false, puts the batch back to `queued`, and tries again after 1.5 s. The edit stays on screen and in the queue.

Nothing is lost while offline: edits pile up in `pending`, applied locally, and go out in order when the server answers again. The number waiting is `unsaved` in the session state, and the editor's header turns it into **Saving…**, **Saved** or, while the server can't be reached, **Offline, will retry**.

A session that may only read (a view link, or the role **viewer**) never flushes: `readOnly` stops it. What that person types stays in their tab and is never sent. If their role is later raised, the pending changes are dropped and the document becomes what the server has, so something typed while only viewing is not suddenly saved.

The stream

The session opens `GET /api/docs/:id/events?client=…&mode=…` as an `EventSource` (through `listen()`, which adds the share token for a guest). See The event stream for what the server sends. The session handles each type:

Event	What the session does
`hello`	If the server's version differs from the session's and nothing is pending, resync.
`ops`	Apply to `confirmed`; see below.
`message`, `presence`, `people`	Update the conversation, who is acting and who has the document open.
`access`, `revoked`	A new role; or the person lost access, so show why and stop the stream.
`data`	Reload the collection if the document uses it.
`fetch`, `trace`	Update a fetch cell's state; add the handlers the server ran to the trace.
`reset`	The document was replaced (sync rewrote its history): `reload()`.
`deleted`, `unshared`	The document is gone, or the link was turned off.
`ping`	Nothing. It only proves the stream is alive.

A **watchdog** runs every ten seconds. If nothing, not even a ping, has arrived for 25 seconds, it opens a new stream and resyncs. A stream can die without telling the browser (a proxy, a server restart), and this is how the page notices.

onOps: rebasing

An `ops` event carries the version it makes, the actor, the ops *as applied*, the cells they touched, and the sender's `client` and `batch`. `onOps` does this:

1. If the version is not newer than `confirmed.v`, ignore it. If it is not exactly `confirmed.v + 1`, a version was missed: `resync()`.
2. Apply its ops to `confirmed` and set the version. If they don't apply, `resync()` too.

3. If the event is **ours** (same `client`), remove the batch with that `batch` id from `pending`. The change is now in `confirmed`. The optimistic copy is kept unless nothing is pending, in which case `state.doc` becomes `confirmed`.
4. If it is **someone else's**, rebuild `state.doc` with `rebase()` and flash the cells it `touched` in that actor's colour for about two seconds.

TypeScript

```
private rebase(): Doc {
  let doc = this.confirmed!;
  for (const p of this.pending) {
    if (p.state === 'done' && (p.v ?? 0) <= this.confirmed!.v) continue;
    try {
      doc = applyOps(doc, p.ops).doc;
    } catch {
      // A change made elsewhere got there first; the server will refuse this one too.
    }
  }
  return doc;
}
```

`rebase` is the heart of it. When someone else's change arrives, the session takes `confirmed` (now including it) and replays everything still pending on top. Edits to different cells both apply. An edit that no longer applies, because the other person removed its cell, is skipped, and the server will refuse it as well when it arrives.

The server is the authority on order. It applies batches in the order they reach it, and gives each a version; every session applies ops in version order. So two browsers that make changes at the same moment may see them in different orders for a few milliseconds, and then both converge on the server's.

resync and reload

`resync()` is the repair: fetch the whole document and the log since `confirmed.v`, drop pending batches the new document already includes (`done` and `v ≤ doc.v`), rebase what remains and reload the collections. It runs after a missed version, a stream that fell silent, a failed apply, and when a slide in a deck is resumed.

`reload()` is for `reset`, when history itself was replaced. It also discards the undo and redo stacks, because their entries refer to a history that no longer exists, and keeps queued batches.

Undo and redo

```TypeScript
interface UndoEntry {
  op: Op;
  key?: string;
  ts: number;
}
```

Each `dispatch` pushes the *inverse* of what it applied (see Applying ops). `undo()` pops an entry and dispatches it with `undo: false` and `react: false`; the inverse of that goes on the redo stack. Dispatching a new change empties the redo stack.

- **Steps.** Ops made in one dispatch, including those their handlers added, share one entry. Undoing a button press undoes what the handlers did too.
- **Keys.** Changes with the same `key` within 1.5 s share an entry (typing in an input, dragging a slider, a divider). The inverse already on the stack is the one from before the first keystroke, so one undo undoes the whole burst. When handlers changed something on a later keystroke, the entry becomes `["do", newInverse, old]` so their effects are undone first.
- **Size.** The stack keeps 200 entries.

Undo is itself an ordinary dispatch: it is applied here, queued, sent and confirmed like any change, and other people see it as a change by you. It does not run handlers (`react: false`), because the inverse already includes whatever the handlers changed.

Undo works on the shared document, not on a private history: it undoes *your* last change even if someone else has changed things since, by applying the inverse made when you did it. If that inverse no longer fits, `applyOps` fails and the toast says why.

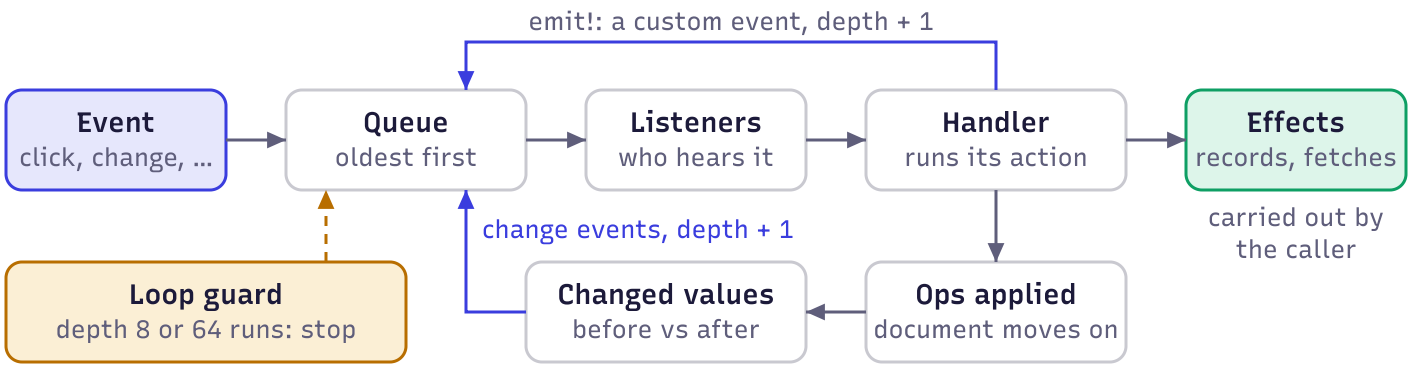
Handlers run in the browser

Events are the other half of `dispatch`. `events.ts` is in `core`, so the browser can do what the server does:

```TypeScript
react(doc, world, start): Reaction   // { doc, ops, effects, trace }
```

`react` takes a document, a world and a starting point (ops a person made, a cell's own action, events) and runs the handlers they set off, each seeing what the ones before

changed. See the event list. It stops at `MAX_DEPTH` (8) and `MAX_RUNS` (64) with an error in the trace.



Listeners: who hears an event

a cell's event (click, change, pick, ...)	that cell's own on, if it has a handler for it
a custom event (emit!)	every cell with a handler for it, then the document's on
open, close	the document's on

Depth 0 is the event itself, 1 an event that a handler caused, and so on.
The guard stops at an event 8 deep or after 64 handlers; what already ran is kept, and the trace says why.

The session calls it from five places. All but the code studio's test run only in **Live**:

Method	What starts it
`dispatch`	A change a person made; handlers for the value changes it caused.
`run(cellId)`	A button, or a table row's action: the cell's own `do` runs as the gesture and, for a button, its `click` handler after it. A failure shows `That button failed: …`.
`fire(event)`	A click or double-click (`pointer()`), and the document's `open`.
`test(cell, event)`	The code studio's button for trying a handler, in any mode. Its changes are real and one undo step.
`leaving()`	The document's `close` handler, when the person goes. Its ops are sent with `keepalive` so they survive the page closing.

The result has three parts, and the session handles each differently:

- **`ops`** are dispatched with `react: false` (they came from handlers, which must not trigger themselves) and join the gesture's undo step.

- **`effects`** are what leaves the document: records to insert, update, delete or clear, and fetches to refresh. `carryOut` sends them to the API one by one, to the document's owner's collections, through the document (`/api/data/<name>?doc=<id>`). A read-only session shows a message and sends nothing.
- **`trace`** is added to the `trace` list shown in Activity, marked `here`.

What the browser does *not* run is everything that happens without a person: timers, fetch cells, and the handlers an agent's change sets off. The server runs those, once per document, and sends the trace back in a `trace` event (marked `server`). See The runner. The two meet through a mode message: `told(mode)` posts POST /api/docs/:id/viewer whenever the tab changes between Edit, Live and Page or the selection settles, and the server keeps timers running while at least one tab is in Live.

Time

If `computed.usesNow` is true, the session runs a one-second clock that patches `now` and so re-evaluates the formulas that read the time (see The engine). The clock stops when no formula needs it, and `pause()` stops it for a slide that is not on screen.

Other things the session owns

- **Selection and editing**: which cells are selected, which is being typed in, and the seed text for the first keystroke, so it isn't lost when typing starts an edit.
- **Commands** shared by the toolbar, keyboard and menus (`split` , `merge` , `duplicate` , `setKind` , `clearOrRemove`), each a thin wrapper that builds ops and dispatches them. `setKind` builds the notation for the new kind, which is why it lives here and not in `core`.
- **Presence**: it reports the selected cell to the server (after the selection has settled for 150 ms) so others see it, and records who acted last.
- **Pause and resume**: a deck's slides that are not on screen close their stream and stop their clock; `resume()` reconnects and resyncs.
